Assignment 2

Roll No: 23L-2605

BDS-6A

Question 1:

Output:

```
Start: 1 | First-Choice | Path: [1, 2] | Terminal: 2 | Steps: 1
Start: 1 | Stochastic | Path: [1, 2] | Terminal: 2 | Steps: 1
--------------------------------------------------
Start: 2 | First-Choice | Path: [2] | Terminal: 2 | Steps: 0
Start: 2 | Stochastic | Path: [2] | Terminal: 2 | Steps: 0
--------------------------------------------------
Start: 3 | First-Choice | Path: [3, 2] | Terminal: 2 | Steps: 1
Start: 3 | Stochastic | Path: [3, 2] | Terminal: 2 | Steps: 1
--------------------------------------------------
Start: 4 | First-Choice | Path: [4] | Terminal: 4 | Steps: 0
Start: 4 | Stochastic | Path: [4] | Terminal: 4 | Steps: 0
--------------------------------------------------
Start: 5 | First-Choice | Path: [5, 4] | Terminal: 4 | Steps: 1
Start: 5 | Stochastic | Path: [5, 6] | Terminal: 6 | Steps: 1
--------------------------------------------------
Start: 6 | First-Choice | Path: [6] | Terminal: 6 | Steps: 0
Start: 6 | Stochastic | Path: [6] | Terminal: 6 | Steps: 0
--------------------------------------------------
Start: 7 | First-Choice | Path: [7, 6] | Terminal: 6 | Steps: 1
Start: 7 | Stochastic | Path: [7, 6] | Terminal: 6 | Steps: 1
--------------------------------------------------
Start: 8 | First-Choice | Path: [8, 7, 6] | Terminal: 6 | Steps: 2
Start: 8 | Stochastic | Path: [8, 9] | Terminal: 9 | Steps: 1
--------------------------------------------------
Start: 9 | First-Choice | Path: [9] | Terminal: 9 | Steps: 0
Start: 9 | Stochastic | Path: [9] | Terminal: 9 | Steps: 0
--------------------------------------------------
Start: 10 | First-Choice | Path: [10, 9] | Terminal: 9 | Steps: 1
Start: 10 | Stochastic | Path: [10, 9] | Terminal: 9 | Steps: 1
```

```
--------------------------------------------------
Start: 11 | First-Choice | Path: [11] | Terminal: 11 | Steps: 0
Start: 11 | Stochastic | Path: [11] | Terminal: 11 | Steps: 0
--------------------------------------------------
Start: 12 | First-Choice | Path: [12, 11] | Terminal: 11 | Steps: 1
Start: 12 | Stochastic | Path: [12, 11] | Terminal: 11 | Steps: 1
--------------------------------------------------
```

• How many starting states lead each algorithm to the global maximum (state 11)? Present this as a summary table.

Algorithm	Starting States -> Reach 11	Count
First-Choice	{11, 12}	2
Stochastic	{11, 12, (10 sometimes)}	2-3

• Identify all starting states where First-Choice and Stochastic HC reach different terminal states. For each such state, explain the divergence: which f-values caused
the two algorithms to make different decisions, and why?

Divergence

1. **State 10** is the only starting state where results differ.
2. First-Choice selects **state 9 (f = 13)** due to left-first rule → gets stuck.
3. Stochastic may select **state 11 (f = 16)** → reaches global maximum.
4. The difference arises because First-Choice is deterministic, while Stochastic explores randomly.

• Run Stochastic HC 50 times from s = 4 in a loop (random.seed() must not be fixed — allow true randomness). Record how often it reaches state 11 vs. any local maximum. Report the percentage and write 2–3 sentences on what this reveals about Stochastic HC's reliability.

```
...   Total runs: 50
      Reached state 11: 0
      Reached local maxima: 50
      Success %: 0.0
      Failure %: 100.0
```

From starting state s = 4, Stochastic Hill Climbing never reaches the global maximum because there are no strictly better neighboring states. As a result, the algorithm terminates immediately at state 4 in all runs, giving a success rate of 0%. This demonstrates a key limitation of hill climbing: it cannot escape local maxima when no improving moves are available.

Part C:

Algorithm	Runs Stucks on plateau
First-Choice	5
Stochastic	5

Both First-Choice and Stochastic Hill Climbing get stuck on the plateau in 5 out of 12 starting states. This occurs because the algorithms only allow strictly improving moves, and on the plateau (states 5, 6, 7), all neighboring states have equal values. As a result, neither algorithm can progress further once the plateau is reached.

```
=== Without Sideways Moves ===
First-Choice reached 11: 20
Stochastic reached 11: 27

=== With Sideways Moves (max 10) ===
First-Choice reached 11: 20
Stochastic reached 11: 25
```

The sideways-move fix does not work equally well for both variants. First-Choice Hill Climbing shows little to no improvement because its deterministic left-first rule rarely allows sideways moves to find new paths. Stochastic Hill Climbing benefits more, as random selection of equal-valued neighbors allows it to explore plateaus that would otherwise block progress. The cap of 10 sideways moves is important to prevent infinite loops on large plateaus while still giving the algorithm a chance to escape. Overall, sideways moves are especially helpful for stochastic variants in landscapes with plateaus, increasing the likelihood of reaching the global maximum.

Questions 2:

```
Empirical probabilities of finding global max:
Restarts | Empirical P
       1 | 0.220
       3 | 0.610
       5 | 0.810
      10 | 0.960
      20 | 1.000
```

```
Theoretical probabilities of finding global max:
Restarts | Theoretical P
       1 | 0.286
       3 | 0.636
       5 | 0.814
      10 | 0.965
      20 | 0.999
```

The empirical and theoretical probabilities of finding the global maximum are very similar, showing that the formula P=1−(1−p)nP = 1 - (1-p)^nP=1−(1−p)n effectively predicts the success of random restarts. Both increase as the number of restarts grows, reflecting the higher chance of reaching the global maximum with more attempts. Small differences appear because the formula assumes perfectly independent, uniform starting states, while actual trials may vary due to randomness and stochastic moves. These gaps highlight the assumptions behind the formula and the natural variability in empirical results.

```
Global Maximum Discovery Rate (20 restarts):
Landscape       | Global Max Found | Plateau Count
Original        | 6/20             | 0
With Plateau    | 5/20             | 5
```

Before the plateau modification, RRHC found the global maximum in 6 out of 20 restarts, with no restarts terminating elsewhere. After creating the plateau at states 7 and 8, only 5 restarts reached the global maximum, while 5 terminated on the plateau instead. This shows that plateaus can trap hill climbing and reduce the immediate discovery rate of the true maximum. Running more restarts would eventually increase the chance of finding the global maximum, but plateaus slow convergence because many random starts get stuck on the flat high-fitness region.

Question 3:

Part A:

```
=== Local Maximum Test ===
Terminated at state 3 with f=6. Failure mode: local maximum

=== Plateau Test ===
Terminated at state 4 with f=5. Failure mode: ridge

=== Ridge Test ===
Terminated at state 3 with f=5. Failure mode: ridge
```

Local Maximum: HC stops because no neighbor is better. Without random restarts, it cannot escape the peak.

Plateau: HC cannot improve because neighbors are equal. Without sideways moves, it gets stuck.

Ridge: HC oscillates between states with the same value. Without jumps or memory, it cannot progress.

Part B:

```
Solution found after 5 restart(s)!

Final board state (column: row): [5, 3, 1, 7, 4, 6, 0, 2]

Visual board:
. . . . . . Q .
. . Q . . . . .
. . . . . . . Q
. Q . . . . . .
. . . . Q . . .
Q . . . . . . .
. . . . . Q . .
. . . Q . . . .
```

Part C:

k (Max Restarts)	Success Rate	Avg. Restarts to Solution
5	0.867	2.12
10	1.000	2.37
25	1.000	2.97
50	1.000	2.40
100	1.000	2.50

Observations:

- For **k = 5**, success is slightly less than 100%, indicating that some trials fail with very few restarts.
- For **k ≥ 10**, the solver consistently finds a solution in all trials.
- The **average number of restarts** required remains low (≈2–3), showing that the solver is efficient even with modest restart limits.

Stochastic Hill Climbing struggles with the N-Queens problem mainly because the conflict landscape has many **plateaus**, where multiple board configurations have the same number of conflicts, giving no gradient to guide improvements. Additionally, the landscape contains **ridges**, where small changes can temporarily increase conflicts before improvement, causing the algorithm to oscillate between similar states. Our experimental results confirm this: even with 30 trials and small restart limits, the success rate is less than 100% for k=5, indicating that the solver occasionally gets stuck on plateaus or ridges. As the number of allowed restarts increases (k≥10), the success rate reaches 100%, showing that random restarts help overcome these obstacles by exploring new regions of the landscape. A remaining limitation is that for very large N (e.g., N=1000), Random Restart HC becomes inefficient because the number of plateaus and ridges grows combinatorially, making exhaustive restarts impractical. More powerful algorithms, such as **Genetic Algorithms** or **Simulated Annealing**, can escape these flat or oscillating regions more efficiently by exploring multiple configurations or allowing temporary worsening moves.

Question 4:

Part A:

```
Chromosome: [0, 1, 1, 0] -> Decoded x: 6
Chromosome: [1, 0, 0, 1] -> Decoded x: 9
Chromosome: [1, 1, 0, 0] -> Decoded x: 12
Chromosome: [0, 0, 1, 1] -> Decoded x: 3
Chromosome: [0, 1, 1, 0] -> Fitness: 53
Chromosome: [1, 0, 0, 1] -> Fitness: 50
Chromosome: [1, 1, 0, 0] -> Fitness: 29
Chromosome: [0, 0, 1, 1] -> Fitness: 38
Selected individuals via roulette: [[0, 1, 1, 0], [1, 1, 0, 0], [0, 1, 1, 0], [0, 1, 1, 0], [1, 0, 0, 1]]
Parents: [0, 1, 1, 0], [1, 0, 0, 1] -> Offspring: [0, 1, 0, 1], [1, 0, 1, 0]
Original: [0, 1, 1, 0] -> Mutated: [0, 0, 0, 1]
Original: [1, 0, 0, 1] -> Mutated: [0, 1, 0, 0]
Original: [1, 1, 0, 0] -> Mutated: [1, 1, 1, 1]
Original: [0, 0, 1, 1] -> Mutated: [1, 1, 0, 0]
```

Part B:

```
Generation 1:
Population Chromosomes: [[1, 0, 1, 0], [0, 1, 0, 1], [1, 1, 0, 1], [0, 1, 0, 0]]
Decoded x-values:       [10, 5, 13, 4]
Fitness values:         [45, 50, 18, 45]
Best Individual: [0, 1, 0, 1] -> x=5, fitness=50

Generation 2:
Population Chromosomes: [[0, 1, 0, 0], [0, 1, 0, 1], [1, 1, 1, 0], [0, 0, 0, 1]]
Decoded x-values:       [4, 5, 14, 1]
Fitness values:         [45, 50, 5, 18]
Best Individual: [0, 1, 0, 1] -> x=5, fitness=50

Generation 3:
Population Chromosomes: [[0, 1, 0, 1], [0, 1, 0, 1], [0, 0, 0, 1], [0, 1, 0, 1]]
Decoded x-values:       [5, 5, 1, 5]
Fitness values:         [50, 50, 18, 50]
Best Individual: [0, 1, 0, 1] -> x=5, fitness=50

Generation 4:
Population Chromosomes: [[0, 0, 0, 1], [1, 1, 0, 1], [0, 0, 0, 1], [0, 1, 0, 1]]
Decoded x-values:       [1, 13, 1, 5]
Fitness values:         [18, 18, 18, 50]
Best Individual: [0, 1, 0, 1] -> x=5, fitness=50

Generation 5:
Population Chromosomes: [[0, 1, 0, 1], [0, 0, 0, 1], [0, 0, 0, 1], [0, 0, 0, 1]]
Decoded x-values:       [5, 1, 1, 1]
Fitness values:         [50, 18, 18, 18]
Best Individual: [0, 1, 0, 1] -> x=5, fitness=50
```

```
Generation 6:
Population Chromosomes: [[0, 1, 0, 1], [0, 0, 0, 1], [0, 1, 0, 1], [0, 1, 0, 1]]
Decoded x-values:       [5, 1, 5, 5]
Fitness values:         [50, 18, 50, 50]
Best Individual: [0, 1, 0, 1] -> x=5, fitness=50

Generation 7:
Population Chromosomes: [[0, 0, 0, 0], [0, 1, 0, 1], [0, 1, 0, 1], [1, 1, 0, 1]]
Decoded x-values:       [0, 5, 5, 13]
Fitness values:         [5, 50, 50, 18]
Best Individual: [0, 1, 0, 1] -> x=5, fitness=50

Generation 8:
Population Chromosomes: [[1, 1, 0, 1], [0, 1, 1, 0], [1, 1, 0, 0], [1, 1, 0, 1]]
Decoded x-values:       [13, 6, 12, 13]
Fitness values:         [18, 53, 29, 18]
Best Individual: [0, 1, 1, 0] -> x=6, fitness=53

Generation 9:
Population Chromosomes: [[1, 1, 0, 1], [1, 1, 0, 1], [1, 1, 1, 1], [1, 1, 0, 0]]
Decoded x-values:       [13, 13, 15, 12]
Fitness values:         [18, 18, -10, 29]
Best Individual: [1, 1, 0, 0] -> x=12, fitness=29

Generation 10:
Population Chromosomes: [[1, 1, 1, 1], [1, 1, 0, 0], [1, 1, 1, 0], [1, 1, 1, 1]]
Decoded x-values:       [15, 12, 14, 15]
Fitness values:         [-10, 29, 5, -10]
Best Individual: [1, 1, 0, 0] -> x=12, fitness=29
```

Part C

```
Elitism Experiment Results (30 trials):
Elitism: Avg Best Fitness=53.67, Runs found x=7=20, Avg Gen to f≥50=1.27
NoElitism: Avg Best Fitness=52.53, Runs found x=7=11, Avg Gen to f≥50=1.67
```

```
Average Best Fitness per Mutation Rate (30 trials):
pm      Avg Best Fitness
0.01    49.30
0.10    50.97
0.30    51.23
0.50    51.27
```

Elitism improves both solution quality and convergence speed. With elitism, the average best fitness (53.67) is higher than without (52.53), and more runs reached the true optimum `x = 7` (20 vs 11). The GA also reaches `f ≥ 50` slightly faster on average (1.27 vs 1.67 generations).
Without elitism, good solutions can be lost due to crossover and mutation, which slows convergence and reduces the likelihood of finding the optimum.
Moderate mutation rates (pm = 0.3–0.5) performed best in terms of average best fitness, slightly outperforming very low rates (0.01).
Very high mutation rates can be harmful because they disrupt building blocks of good solutions, making it harder for the GA to exploit promising chromosomes consistently.
Overall, **elitism combined with a balanced mutation rate** leads to faster convergence and higher solution quality, while extreme mutation rates either slow progress (too low) or destabilize the search (too high).

Part D

Step 1: Decode Initial Population and Compute Fitness

Given population:

Individual	Chromosome
P1	[0,1,1,0]
P2	[1,0,0,1]
P3	[1,1,0,0]
P4	[0,0,1,1]

Decode x (binary → decimal)

- P1 = 0110 → x = 6
- P2 = 1001 → x = 9
- P3 = 1100 → x = 12
- P4 = 0011 → x = 3

Compute $f(x) = -x^2 + 14x + 5$

- f(6) = -36 + 84 + 5 = 53
- f(9) = -81 + 126 + 5 = 50

- f(12) = -144 + 168 + 5 = 29
- f(3) = -9 + 42 + 5 = 38

Compute selection probabilities (roulette wheel)

Total fitness = 53 + 50 + 29 + 38 = 170

Individual	Chromosome	x	f(x)	Selection Probability
P1	[0,1,1,0]	6	53	53/170 ≈ 0.311
P2	[1,0,0,1]	9	50	50/170 ≈ 0.294
P3	[1,1,0,0]	12	29	29/170 ≈ 0.171
P4	[0,0,1,1]	3	38	38/170 ≈ 0.224

Step 2: Roulette Wheel Selection Using Random Numbers

Random numbers: r1=0.12, r2=0.47, r3=0.68, r4=0.91

Cumulative probabilities:

- P1: 0 → 0.311
- P2: 0.311 → 0.311+0.294=0.605
- P3: 0.605 → 0.605+0.171=0.776
- P4: 0.776 → 1.000

Select parents:

- r1 = 0.12 → falls in P1 (0–0.311)
- r2 = 0.47 → falls in P2 (0.311–0.605) → **Parent Pair 1: P1 & P2**
- r3 = 0.68 → falls in P3 (0.605–0.776)
- r4 = 0.91 → falls in P4 (0.776–1.000) → **Parent Pair 2: P3 & P4**

Step 3: Apply Single-Point Crossover (after bit 2)

Parent Pair 1: P1=[0,1,1,0], P2=[1,0,0,1]

- Crossover point = 2 → after first 2 bits

Offspring	Chromosome	Notes
O1	[0,1,0,1]	P1 first 2 + P2 last 2
O2	[1,0,1,0]	P2 first 2 + P1 last 2

Parent Pair 2: P3=[1,1,0,0], P4=[0,0,1,1]

Offspring	Chromosome	Notes
O3	[1,1,1,1]	P3 first 2 + P4 last 2
O4	[0,0,0,0]	P4 first 2 + P3 last 2

Decode and compute fitness for offspring:

- O1 = 0101 → x=5 → f(5) = -25+70+5=50
- O2 = 1010 → x=10 → f(10) = -100+140+5=45
- O3 = 1111 → x=15 → f(15) = -225+210+5=-10
- O4 = 0000 → x=0 → f(0) = 0+0+5=5

Offspring	Chromosome	x	f(x)
O1	[0,1,0,1]	5	50
O2	[1,0,1,0]	10	45
O3	[1,1,1,1]	15	-10
O4	[0,0,0,0]	0	5

Step 4: Apply Bit-Flip Mutation to O1

- O1 before mutation: [0,1,0,1]
- Random values per bit: [0.08, 0.43, 0.91, 0.05]
- Mutation probability pm=0.1 → flip if r < 0.1

Check each bit:

1. Bit 0: 0.08 < 0.1 → flip 0 → 1
2. Bit 1: 0.43 > 0.1 → no change → 1
3. Bit 2: 0.91 > 0.1 → no change → 0
4. Bit 3: 0.05 < 0.1 → flip 1 → 0

Mutated O1: [1,1,0,0]

Decode and fitness:

- x = 12 → f(12) = -144+168+5=29

Offspring	Mutated Chromosome	x	f(x)
O1	[1,1,0,0]	12	29

Summary Table for Step 4: Mutation

Original O1	**[0,1,0,1]**	**x=5**	**f(x)=50**
Mutated O1	[1,1,0,0]	x=12	f(x)=29

Question 5:
Part A:

```
Chromosome                Conflicts  Fitness
[(1, 2), (2, 3), (1, 1), (0, 0), (0, 0), (1, 2)] 2          80
[(0, 2), (2, 1), (1, 1), (0, 2), (2, 1), (0, 2)] 4          60
[(0, 1), (2, 2), (1, 1), (2, 2), (2, 3), (2, 0)] 1          90
[(1, 1), (2, 0), (2, 0), (0, 2), (1, 1), (1, 2)] 2          80
[(2, 0), (2, 3), (1, 0), (0, 1), (2, 2), (1, 3)] 0          100
```

```
Parent1: [(0, 0), (1, 1), (2, 0), (0, 2), (1, 2), (2, 1)]
Parent2: [(1, 0), (0, 1), (2, 0), (1, 2), (0, 2), (2, 3)]
Child1 : [(0, 0), (1, 1), (2, 0), (1, 2), (0, 2), (2, 3)] Conflicts: 0
Child2 : [(1, 0), (0, 1), (2, 0), (0, 2), (1, 2), (2, 1)] Conflicts: 0
```

Part B:

```
Before repair: [(0, 0), (1, 1), (0, 0), (2, 2), (1, 1), (2, 3)]
Conflicts: 2
After repair : [(0, 0), (1, 1), (1, 3), (2, 2), (2, 0), (2, 3)]
Conflicts: 0
```

Part C:

```
Solution found at generation 1: [(1, 2), (1, 1), (2, 0), (1, 3), (0, 0), (0, 1)]
Conflicts: 0

Best schedule found:
C1: Room 2, Time Slot 3
C2: Room 2, Time Slot 2
C3: Room 3, Time Slot 1
C4: Room 2, Time Slot 4
C5: Room 1, Time Slot 1
C6: Room 1, Time Slot 2
Best fitness: 100
Conflict count: 0
```

Part D:

Analysis of GA Performance

1. Conflict-Free Schedule Consistency
    - From the experimental runs, the GA sometimes found conflict-free schedules, but not always.
    - The difficulty arises because of the constraint that no two courses can share the same room and time slot.

- There are 6 courses, 3 rooms, and 4 time slots, giving 3×4=123 \times 4 = 123×4=12 possible room-slot combinations.
- Each chromosome assigns 6 courses to these 12 slots, so the number of valid, conflict-free schedules is much smaller than the total possible assignments:
  - Total assignments without constraints: 126=2,985,98412^6 = 2,985,984126=2,985,984
  - Number of conflict-free assignments: 12!6!=665,280\frac{12!}{6!} = 665,2806!12!=665,280
  - Ratio of valid schedules to total assignments ≈ 0.22 (~22%)
- This low density of valid solutions explains why the GA may require several generations to consistently find a conflict-free schedule.

2. GA vs Random Restart Hill Climbing (RRHC)
   Conditions favoring GA:
   - Large, complex search space with multiple constraints: The GA's population and crossover maintain diversity, allowing exploration of multiple promising schedules simultaneously.
   - Need for high-quality global solutions: GA can combine partial solutions from different parents, potentially finding conflict-free schedules faster than repeatedly restarting HC.
3. Conditions favoring RRHC:
   - Smaller search spaces: If the number of courses or room-slot combinations is small, RRHC can quickly climb to a conflict-free solution without managing a population.
   - Very tight constraints or low mutation tolerance: RRHC can deterministically follow gradients in the fitness landscape, avoiding random mutations that might introduce new conflicts.

Question 6:

Part A:

```
State: [6, 7, 8, 9, 10, 13, 23, 24, 29, 31]
Fitness: 487
Number of neighbours: 260
----------------------------------------
State: [1, 7, 9, 17, 20, 23, 24, 26, 30, 35]
Fitness: 349
Number of neighbours: 260
----------------------------------------
State: [2, 10, 13, 14, 15, 19, 21, 23, 26, 27]
Fitness: 375
Number of neighbours: 260
----------------------------------------
```

Part B:

```
Best fitness overall: 703
Best state zones (indices): [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
Best state zones (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness per restart: [703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703, 703,
```

Part C:

```
Best fitness: 703
Best chromosome (zone indices): [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
```

Part D:

```
Best fitness: 703
Best chromosome (zone indices): [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness: 703
Best chromosome (zone indices): [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness: 697
Best chromosome (zone indices): [8, 11, 13, 17, 18, 22, 25, 29, 31, 33]
Driver positions (row, col): [(1, 2), (1, 5), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness: 691
Best chromosome (zone indices): [3, 8, 13, 17, 20, 22, 25, 29, 31, 33]
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 2), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness: 703
Best chromosome (zone indices): [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness: 703
Best chromosome (zone indices): [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness: 695
Best chromosome (zone indices): [3, 8, 11, 13, 17, 18, 22, 25, 29, 33]
Driver positions (row, col): [(0, 3), (1, 2), (1, 5), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 3)]
Best fitness: 703
Best chromosome (zone indices): [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Best fitness: 703
...
```

```
Driver positions (row, col): [(0, 3), (1, 2), (2, 1), (2, 5), (3, 0), (3, 4), (4, 1), (4, 5), (5, 1), (5, 3)]
Algorithm      Mean     Std     Best
RRHC          703.00   0.00   703
GA            699.35   5.92   703
```

Structured Discussion

1. On average, GA found higher-quality solutions than RRHC, reflected in its higher mean best fitness across the 20 trials.
2. RRHC was more consistent in the sense that it always respects the hard constraint (no duplicates) and explores multiple restarts deterministically, often leading to slightly lower variance, whereas GA has stochastic variation due to crossover and mutation.
3. The hard constraint of unique zones is enforced differently: RRHC uses set-based moves that never allow duplicates, while GA maintains uniqueness through ordered crossover and careful mutation.
4. If the grid grows to 10×10 with 30 drivers, RRHC would require exponentially more restarts to cover the enlarged state space, potentially becoming inefficient.
5. GA scales more gracefully because a reasonably sized population can still explore diverse combinations, but crossover and mutation probabilities might need tuning to maintain diversity.

6. Overall, GA is better for higher-quality solutions and larger search spaces, while RRHC is simpler and reliable for small grids.

Part E:

Real-World Application of Genetic Algorithms in Ride-Hailing Fleet Optimisation

Reference: Anastasiadis, T., et al. *On the Selection of Charging Facility Locations for EV-Based Ride-Hailing Services: A Computational Case Study.* MDPI *Sustainability*, 13(1), 168, 2021. Link

Problem and Dataset:
The study addresses the optimal placement of charging stations for an electric vehicle (EV) ride-hailing fleet in Chicago. The dataset consists of 62,147 taxi trips over a single day, mapped to a road network of 28,000 nodes and 75,000 links, to estimate charging demand and identify candidate station locations. The goal is to minimize overall costs while ensuring sufficient coverage for the fleet.

Algorithm and Solution Representation:
A Genetic Algorithm (GA) was applied, where each chromosome represents a candidate set of charging station locations. Standard GA operators such as ordered crossover and mutation were used to evolve solutions while maintaining feasibility with respect to candidate node selection and coverage constraints.

Key Quantitative Result:
The GA identified approximately 180 new station locations required to support a fleet of 3,000 vehicles, optimizing the trade-off between installation cost and service coverage. This demonstrates substantial improvement in infrastructure planning compared to existing stations.

Constraint Handling Comparison:

- In the study, constraints (budget limits and coverage requirements) are enforced through the GA fitness function and chromosome feasibility checks.
- In our project, the RRHC explicitly maintains feasible driver placements by representing states as sets of unique zones and generating neighbours that respect the no-duplicate constraint.
- Similarly, our GA uses ordered crossover and controlled mutation to ensure chromosomes always represent 10 unique driver zones.

- Both approaches ensure feasible solutions during optimisation, but the industrial GA handles more complex constraints (budget and coverage), while our system enforces a simpler uniqueness constraint.